



Uzhavan Direct

Farm-to-Table, Direct. A zero-commission agricultural marketplace with a three-tier FPO logistics backbone.

Static SPA

3 Roles

3 Languages

3 Themes

FPO 3-Tier Portal

Express + PostgreSQL API

Feature & System Report

Smart India Hackathon 2026 · Repository: `uzhavandirect` · Branch `main`

Report generated: 1 October 2026

This document is a complete inventory of what is present in the Uzhavan Direct codebase: the architecture, every role and view, the full feature catalogue module by module, the order-tracking pipeline, the FPO logistics portal, the data model, the UI/UX systems, and the accompanying backend API — followed by an honest list of current limitations.

1 · Project Snapshot

3

USER ROLES

25

FRONTEND VIEWS

8

DELIVERY STAGES

3

TAMIL / HINDI / ENGLISH

25/25

API TESTS PASSING

Layer	Technology	Notes
Frontend	Vanilla JavaScript (ES6), HTML5, CSS3	No framework, no bundler, no build step. Single <code>app.js</code> (~3,425 lines) drives all state, routing and rendering.
Maps	Leaflet 1.9.4 (CDN)	Live tracking map inside the order-tracking modal; Leaflet CSS + JS loaded from unpkg.
Fonts	Google Fonts — DM Sans, Fraunces	Loaded via <code>fonts.googleapis.com</code> .
Persistence (frontend)	<code>localStorage</code>	All demo data lives in the browser behind a small <code>store</code> facade.
Backend	Node.js, Express, PostgreSQL	<code>server/</code> — JWT auth, bcrypt, OTP flow, FPO batches. Currently runs standalone (not yet wired to the SPA).
Tests	Integration suite over <code>pg-mem</code>	25 passing tests covering auth, produce, auctions, orders and FPO batches.

Repository layout

Path	Contents
<code>index.html</code>	Auth screen + app shell (sidebar, topbar, cart drawer, quantity modal, view root).
<code>app.js</code>	i18n, themes, seed data, store, auth, routing, all views and actions for farmer / consumer / FPO.
<code>style.css</code>	Design tokens, layout, components, FPO portal styles, dark mode, responsive rules.
<code>assets/products/</code>	15 bundled produce photographs (rice, wheat, tomato, onion, potato, carrot, brinjal, spinach, mango, banana, coconut, milk, eggs, turmeric, toor dal).
<code>assets/hero-farmer.webp</code>	Hero imagery for the auth screen.
<code>assets/CREDITS.md</code>	Attribution for bundled imagery.
<code>server/</code>	Express + PostgreSQL API: <code>src/routes</code> , <code>src/middleware</code> , <code>schema.sql</code> , <code>test/api.test.js</code> .
<code>.nojekyll</code>	Disables Jekyll processing for static hosting (GitHub Pages).

2 · Architecture & Application Flow

Uzhavan Direct is a **single-page application with imperative rendering**. There is no virtual DOM: each view is a function that receives the `#viewRoot` element, clears it, and builds DOM nodes (mostly via `innerHTML` templates plus `document.createElement` for interactive parts).

Boot sequence

1. **Seed** — `seed()` populates demo users, produce, orders, auctions and demand (guarded once by `ud_seeded_v2`).
2. **Migrations** — `ensureConsumerProfileFields()`, `ensureDemoCredentials()` (`ud_demo_credentials_v2`), and `ensureFpoPortalData()` (`ud_fpo_v1`) are non-destructive and each run only once.
3. **Auth gate** — `ud_currentUser` is deleted on load and `currentUser = null`, so a fresh login is required on every page visit.
4. **Theme & language** — saved `ud_theme`, `ud_color_mode` and `ud_lang` are re-applied before first paint.
5. **Shell render** — `boot()` unhides `#appShell`, resolves the role from the user record, builds the role-specific sidebar via `renderNav()`, and draws the dashboard via `renderView('dashboard')`.

Routing model

Routing is a two-level dispatch: `renderNav()` picks the sidebar array for the current role, and `renderView(viewId)` picks the matching render function from a per-role map. Views that are not in the sidebar (profile, settings, sub-lists) still resolve through `EXTRA_VIEW_TITLES` so the topbar title is never a raw id.

`goTo(viewId) → renderNav() + renderView(viewId) → render<Role><View>(root)`

3 · Roles & Authentication

Three roles share one auth screen. A segmented role switch selects the intended account type, and the login form validates that the entered credentials actually match that role.

Role	Demo account	Password	Purpose in the system
farmer	karthik_farms / meena_agro	123 / pass123	Lists produce, runs auctions, sees market demand, uses the AI advisor, manages sales.
consumer	divya_buys	123	Shops, bids, tracks deliveries on the map, follows and rates farmers, donates.
fpo	thanjai_fpo	123	Runs first-mile aggregation, corridor batching and destination-hub dispatch.

A single universal demo shortcut exists: entering `john / 123` logs in as whichever role is currently selected — the app maps the shorthand through `DEMO_LOGIN_USERS` to the correct role account.

Authentication features

- **Role-aware login** — credentials are matched, then the record's `type` must equal the selected role.
- **Username or phone login** — the identifier field accepts either.
- **Sign-up with demo OTP** — name, username, 10-digit phone and password; an on-screen demo OTP must be verified before the account is created.
- **Role-specific profile fields** — farmers get village/bio; consumers get city/address; FPOs get source `cluster` and destination `hub`.
- **Session feedback** — a user chip shows the display name; a logout button returns to the auth screen.
- **Password migration** — `ensureDemoCredentials()` normalises the older `meena_agro` password so all demo logins behave predictably.

4 · Complete Feature Catalogue

4.1 · Farmer module

Dashboard

- **Stat strip** — Pending Sales, Completed Sales, Total Income (completed order value *plus* donations received). Each tile is click-through to its detail view.
- **“Your produce listings”** — grid of the farmer's own listings with photo, category, stock, per-unit price and a freshness badge (“Keeps ...”), or a friendly empty state.

Sell Produce

- Create a listing with name, category (`grains/cereals/pulses`, `vegetable`, `fruit`, `dairy`), quantity, unit and price.
- **Automatic enrichment** — `iconFor()` assigns an emoji, `guessPerishability()` computes a shelf-life label, and `productImageUrl()` resolves a bundled photo for 15 known products.

- **Photo upload with client-side compression** — `compressProductImage()` downscales an uploaded image before it is stored as a data URL.

Auctions & bidding

- Farmer-created auctions carry a base rate, quantity, unit and an automatic end time.
- Live count-down timers via `tickTimers()`; `closeAuctionIfNeeded()` closes an auction when it expires.
- **Highest-bid tracking** — shows the current highest bid and bidder; `placeBid()` validates that a new bid beats the standing one.

Market Demand

- Bar-style visualisation of historical demand across ~10 commodities (rice, tomato, onion, milk, mango, carrot, toor dal, banana, potato, eggs).

Farm Advisor (AI chat)

- Conversational assistant with suggested prompts (“Today’s market rate for tomato”, “What crop should I grow?”, “How do I list a new produce?”, “Help”).
- **Rule-based reply engine** — `craftChatReply()` handles greetings, keyword-matched market rates from `MOCK_RATES`, crop guidance and listing help; unknown input falls back gracefully.

Sales management

- **Pending Sales** — per-order rows with a status control and an **“Advance tracking”** button that moves the order one step along the delivery pipeline.
- **Completed Sales** — delivered/closed orders.
- **Money Details** — a ledger view of income, including donations received.

Farmer profile & settings

- Profile card (name, village, bio, rating average) plus follower count.
- Settings: appearance (bright/dark), colour theme picker and language switch — shared with all other roles.

4.2 · Consumer module

Dashboard

- **Stat strip** — Purchased Items (active, non-refunded, undelivered orders) and Total Spent (purchases *plus* donations made).
- **“Fresh from nearby farmers”** — the first four available listings, each with an **Add to cart** button.

Nearby Produce & Search

- **Nearby Produce** — full masonry-style grid of all unsold listings; clicking an item opens the quantity picker.
- **Farmer browse** — per-farmer produce views (`showFarmerProduce`).
- **Search Produce** — searches across produce and farmers; results split into a produce list and a farmer result-card list with an explicit empty state.

Basket, quantity picker and checkout

- **Quantity picker modal** — ± stepper, live unit label and a running total (`Add to cart · ₹...`) before anything is added.
- **Cart drawer** — slide-in panel with line items, per-line quantity changes, remove buttons, a live subtotal, and a delivery hint (“Direct from local farmers · No middlemen commission”).
- **Cart badge** in the topbar keeps the item count visible at all times.
- **Animations & sound** — `animateAddToCart()` flies the product image toward the cart; `playCartPopSound()` and `playCheckoutChime()` provide Web Audio feedback (generated, no audio files).
- **Address gate** — checkout is blocked until the consumer’s city/address is complete, with a redirect into profile settings to fix it, and the cart is resumed afterwards.

Payment modal (demo)

- Four methods: Google Pay, Credit/Debit card, UPI, Cash on Delivery.
- A **demo OTP** is generated per modal open and displayed on screen; “Verify & Pay” checks the entered code, then `completeCheckout()` converts each cart line into a real order and reduces listing stock.

Order tracking & history

- **Purchased Items** — active orders grouped by farmer/product with live stage badges.
- **Purchase History** — past orders, with a rate-and-review entry point.
- **Track Delivery modal** — see §5.
- **Money Details** — spend ledger including donations.

Social, ratings & donations

- **Follow / unfollow farmers** — a “Farmers You Follow” view mirrors `followers[]` on the farmer record.

- **Farmer public profile** — bio, village, produce and aggregate star rating.
- **Rating modal** — 1–5 stars plus an optional comment; re-rating a farmer updates the existing entry rather than duplicating it (`recordRating`).
- **Donation modal** — a consumer can donate to a farmer; donations are written to a shared ledger (`ud_donations`) and appear in both the donor's "Total Spent" and the recipient's "Total Income".
- **Notification bell** — unread badge plus a dropdown; used for donation alerts *and* FPO logistics alerts.

4.3 · FPO Manager module new

See §6 for the full three-tier breakdown. In brief: a logistics dashboard, a first-mile collection and grading table, a corridor batching engine, and a destination-hub de-batching screen with EV last-mile dispatch.

5 · Order Tracking Pipeline

Every order carries a single `deliveryStage`. That one field drives **three surfaces at once** — the farmer's order-management view, the consumer's purchased-items list, and the consumer's tracking timeline — so the three can never disagree.

#	Stage	Meaning
1	<code>placed</code>	Order placed by the consumer; awaiting farmer confirmation.
2	<code>confirmed</code>	Farmer has confirmed the sale.
3	<code>packed</code>	Produce harvested/packed and ready to leave the farm.
4	<code>atFarmerCity</code>	Order has reached the farm-side city / collection point — the entry point for FPO aggregation.
5	<code>atCustomerCity</code>	Long-haul leg complete; order is in the customer's city.
6	<code>atFpo</code>	Order has arrived at the FPO destination hub.
7	<code>outForDelivery</code>	Handed to a last-mile (EV) delivery agent.
8	<code>delivered</code>	Delivered — order status is simultaneously set to <code>completed</code> .

The tracking modal

- **Vertical timeline** — Amazon-style progression with done / current / upcoming states and per-stage descriptions.
- **Leaflet live map** — markers for the farm, intermediate cities and the destination hub, with the route drawn between them.
- **Route HUD** — road distance (km), estimated travel time, and the city route (e.g. *Salem → Chennai*), computed haversine-style from `CITY_COORDINATES`.
- **Delivery-schedule form** — once an order reaches the hub, the consumer picks a time slot (Morning / Afternoon / Evening / Custom date+time) which is persisted as `fpoDeliveryTiming`.
- **Advance Tracking** — steps the order one stage forward with a toast confirmation.
- **Live subscription** — the modal registers a `ud-stage-changed` listener, so if an FPO (or farmer) advances the order while the modal is open, the timeline re-renders and the map re-pans *without a reload*. The listener is detached on close to avoid leaks.

The single writer

`setOrderStage(orderId, stage, extraFields)` is the one function that mutates a delivery stage. It:

1. persists the new `deliveryStage`;
2. sets `status = 'completed'` when the stage is `delivered`;
3. optionally co-writes extra fields (e.g. `fpoDispatchedAt`); and
4. dispatches a `ud-stage-changed` custom event on `window`.

Both the farmer's `advanceOrderStage()` and the legacy `updateOrderStatus()` now delegate to it, which is what makes cross-role live updates possible.

Legacy-data migration

Older orders have no `deliveryStage`. `getOrderStage()` derives one from `status` and rewrites two historic names — `atFarmerFpo` → `atFarmerCity` and `atCity` → `atCustomerCity` — so seeded data keeps working.

6 · FPO Manager Portal — Three Logistics Tiers

The FPO portal layers a producer-organisation logistics network on top of the shared order model. It is available to the `fpo` role and is organised as three sequential tiers, each mapped 1:1 to a sidebar entry.

6.1 · Logistics dashboard

- **Stat strip** — Pending Tier-1 aggregations (lots awaiting grading), Total batch weight in **Tonnes**, Active transit corridors, and Hub delivery queues.
- **Three tier cards** — a visual pipeline (Tier-1 → Tier-2 → Tier-3) with live throughput counts, each clickable straight into its tier.
- **Batch activity list** — recent corridor shipments with corridor name, status chip (Assembling / In transit / Received at hub), vehicle type, weight and order count.

6.2 · Tier-1 — First-Mile Aggregation & AGMARK Grading

Handles produce arriving at village farm-gate collection points.

Element	Behaviour
Awaiting-grading table	Every order at <code>atFarmerCity</code> with no grade yet: order id, source farmer & village, produce, and the ordered quantity.
Digital weighment logger	A numeric input per row (<code>.fpo-weigh-input</code>) captures the actual weighed quantity, independent of the ordered quantity.
AGMARK quality picker	Three grades — A · Premium , B · Standard , C · Processing — selected per row.
Confirm grading	<code>confirmFpoGrade()</code> validates that both a weight and a grade are set, then stamps <code>fpo</code> , <code>fpoGrade</code> , <code>fpoWeighKg</code> and <code>fpoGradedAt</code> and moves the order into the batching queue.
En-route rows	Orders still in <code>packed</code> are listed separately so the manager can see inbound volume.
Graded & queued table	Graded lots with a colour-coded grade badge, weight, and either "Awaiting corridor batch" or the batch id they joined.

6.3 · Tier-2 — Consolidated Freight Corridor Batching (LTL)

Consolidates many small lots into one shipment — Less-Than-Truckload aggregation.

- **Corridor selector** — preset Tamil Nadu freight routes: Nilgiris → Thanjavur, Salem → Chennai, Erode → Coimbatore, Trichy → Madurai.
- **Vehicle selector** — *3-Tonne shared truck* (3,000 kg) or *Kisan Rail parcel van* (5,000 kg).
- **Live capacity meter** — as lots are ticked, a progress track and label show loaded weight against vehicle capacity, and turns to an over-capacity warning state.
- **Queue list** — every graded-but-unbatched lot with a checkbox, id, item and weighment.
- **Create batch** — `createFpoBatch()` requires a selection, rejects loads exceeding vehicle capacity, then writes a shipment record (`status: assembling`) and stamps each order with `fpoBatchId` , `fpoCorridor` and `fpoVehicle` .
- **Dispatch** — `dispatchFpoBatch()` flips the batch to `dispatched` and advances every batched order to the `atFpo` stage in one action, recording `fpoDispatchedAt` .
- **Batch cards** — each shipment shows corridor, vehicle, weight, order count, status chip and its dispatch control.

6.4 · Tier-3 — Destination Hub De-batching & Last-Mile Dispatch

- **Incoming shipments** — every dispatched corridor shipment with a **Receive** button; receiving (`receiveFpoBatch()`) sets the batch to `received` and stamps `receivedAt` .
- **Hub delivery queue** — orders at `atFpo` awaiting doorstep delivery.
- **Consumer delivery slots** — each row shows the slot the *consumer* chose in the tracking modal (e.g. "Morning (8:00 AM–11:00 AM)", or an explicit "Awaiting consumer slot" chip).
- **EV agent assignment** — a dropdown of EV doorstep agents (Kumar · EV-01, Anitha · EV-02, Selvam · EV-03).
- **Guard rail** — assigning an agent is blocked until that order's corridor shipment has been received, with the toast "Receive the corridor shipment first".
- **Assign & dispatch** — `assignFpoEvAgent()` stamps `fpoEvAgent` / `fpoEvAssignedAt` and advances the order to `outForDelivery` .
- **Last-mile list** — orders already out with an EV agent, so the hub can see its live doorstep workload.

6.5 · Consumer notification on every FPO action

Each FPO transition calls `notifyOrderConsumer()` , which appends a message to the consumer's `notifications[]` array and refreshes the notification-bell badge — so grading, dispatch and EV assignment all surface as alerts on the consumer's side.

End-to-end FPO flow

`atFarmerCity · ungraded` → weigh + AGMARK grade → `batch queue` → tick lots → create batch → `assembling` → dispatch → `atFpo` → receive → `received` → assign EV agent → `outForDelivery` → `delivered`

Every arrow above is a `setOrderStage()` call, so the consumer's open tracking modal updates live at each step.

6.6 · FPO profile & settings

- **Profile** — organisation card plus an operations snapshot (lots handled, batches created, total tonnage).
- **Settings** — the shared appearance/theme/language panel plus cluster and hub profile fields.

7 · Data Model (localStorage)

All frontend state is accessed through the `store` facade (`get` / `set` over `localStorage`), which centralises key names and default values.

Key	Shape	Purpose
<code>ud_users</code>	Object of user records	All accounts — type, name, phone, password, profile fields, <code>notifications[]</code> , <code>followers[]</code> .
<code>ud_produce</code>	Array	Farmer listings with category, stock, price, unit, icon, photo, perishability and sold flag.
<code>ud_orders</code>	Array	The multi-role spine: buyer, farmer, item, qty, price, address, <code>status</code> , <code>deliveryStage</code> , FPO fields and delivery timing.
<code>ud_auctions</code>	Array	Auction lots with base rate, quantity, highest bid/bidder, end time and sold flag.
<code>ud_demand</code>	Object	Crop → demand score, used by the market-demand chart.
<code>ud_cart_<username></code>	Array	Per-consumer basket, so carts never collide between accounts.
<code>ud_donations</code>	Array	Shared donation ledger read by both “Total Spent” and “Total Income”.
<code>ud_ratings</code>	Array	Consumer → farmer ratings; one entry per pair, updated on re-rate.
<code>ud_fpo_batches</code>	Array	new FPO shipments: corridor, vehicle, capacity, attached order ids, status and timestamps.
<code>ud_lang</code> · <code>ud_theme</code> · <code>ud_color_mode</code>	String	User preferences (language, palette, light/dark).
<code>ud_seeded_v2</code>	Flag	One-shot demo seed guard.
<code>ud_demo_credentials_v2</code>	Flag	One-shot demo-password normalisation guard.
<code>ud_fpo_v1</code>	Flag	new One-shot, non-destructive FPO migration + demo seed guard.

FPO order fields

Orders touched by the FPO portal gain: `fpo`, `fpoGrade` (A/B/C), `fpoWeighKg`, `fpoGradedAt`, `fpoBatchId`, `fpoCorridor`, `fpoVehicle`, `fpoDispatchedAt`, `fpoEvAgent`, `fpoEvAssignedAt` and `fpoDeliveryTiming`.

Demo data seeded for the portal

- The FPO account `thanjai_fpo` (“Thanjavur Agri Cluster FPO”), with cluster *Nilgiris cluster* and hub *Thanjavur hub*.
- Four orders covering every tier: two ungraded lots at the collection point, one packed/en-route lot, and one already-graded, batched and dispatched lot with a consumer-chosen delivery slot.
- One dispatched corridor batch, so Tier-3 is populated on first login.

8 · UI / UX Systems

Design tokens

CSS custom properties drive colour, radius and shadow throughout: `--forest`, `--forest-dark`, `--turmeric-dark`, `--paper`, `--card`, `--line`, `--ink-soft`, `--danger`, `--success`, `--radius-sm/md`, `--shadow`.

Theming

- **Three palettes** — Harvest (default), Emerald and Teal — applied via a `data-theme` attribute on `<html>`.
- **Light & dark mode** — a separate `data-color-mode`-style switch covering every component (FPO surfaces included).
- **Theme-aware browser chrome** — the `<meta name="theme-color">` tag is updated whenever the palette changes, so mobile browser UI matches.
- **Theme picker** — swatch menu with the palette name and a preview of its four colours.

Localisation

Full three-language UI dictionary (English, Tamil, Hindi) covering navigation, dashboard labels, orders, money, appearance and all FPO tier names. `t(key)` resolves the active language with an English fallback, and the brand name and tagline are localised too (e.g. உழவர் சந்தை / किसान

ਸੰਭੀ).

Interaction & feedback

- **Toasts** — a single reusable `toast()` surface used for successes, guards and errors.
- **Modals** — quantity picker, payment, tracking, rating and donation overlays, all built on a shared `.modal-overLay` pattern with ARIA roles and labelled close buttons.
- **Web Audio feedback** — synthesised cart-pop and checkout-chime sounds; no audio assets required.
- **Fly-to-cart animation** — the product image animates into the cart button on add.
- **Status & grade badges** — colour-coded chips for delivery stages and AGMARK grades.
- **Capacity meter** — a live fill track that warns when a batch exceeds vehicle capacity.

Layout & responsiveness

- Fixed sidebar + scrolling main area with a collapsible mobile drawer (`menuToggle`).
- Responsive produce grid, stat strip that wraps to two columns under 900 px, and full-width form controls on small screens.
- Card-based composition throughout: `.card`, `.section-head`, `.order-row`, `.empty-state`.
- Motion is honoured: a `prefers-reduced-motion` block disables transitions for users who ask for it.

Empty states

Every list view ships a friendly empty state with a glyph and a call to action (no listings, no orders, no search results, no batches), so no screen is ever blank.

9 · Backend API (server/)

A complete Express + PostgreSQL service sits alongside the SPA. It is not yet connected to the frontend — it runs standalone with its own integration suite — and it mirrors the same order pipeline and FPO concepts as the client.

Endpoint	Method	Purpose
<code>/api/auth/register</code>	POST	Create a farmer, consumer or FPO account.
<code>/api/auth/login</code>	POST	Credential login returning a JWT.
<code>/api/auth/me</code>	GET	Current authenticated user profile.
<code>/api/auth/request-otp</code>	POST	Issue a phone OTP (stored in <code>otp_codes</code>).
<code>/api/auth/verify-otp</code>	POST	Verify an OTP and complete registration.
<code>/api/produce</code> · <code>/api/produce/:id</code>	GET / POST / PUT	Browse, create and update produce listings.
<code>/api/auctions</code> · <code>/api/auctions/:id</code>	GET / POST	List auctions and create new lots.
<code>/api/auctions/:id/bid</code>	POST	Place a bid with validation against the standing highest bid.
<code>/api/orders</code> · <code>/api/orders/:id</code>	GET / POST / PATCH	Create and read orders; patch order fields.
<code>/api/orders/:id/stage</code>	PATCH	Server-side equivalent of the client's <code>setOrderStage()</code> — advances an order along the pipeline with FPO stage-band rules.
<code>/api/fpo/batches</code>	GET / POST	List this FPO's shipments; create one — attaching orders assigns them to the FPO.
<code>/api/fpo/batches/:id</code>	GET / PATCH	Read or update a batch's status, details and attached orders.

Database schema

Six tables: `users`, `otp_codes`, `produce`, `auctions`, `orders` and `fpo_batches`.

Server-side safeguards worth noting

- **JWT authentication** with role-based route guards (`requireRole('farmer' | 'fpo' ...)`).

- **bcrypt** password hashing (contrast with the frontend demo, which stores plaintext).
 - **Ownership checks before writes** — e.g. `assertAttachable()` rejects orders that belong to another FPO with a `409 ORDERS_NOT_ATTACHABLE` error listing the offending ids.
 - **Transactional batch attachment** — `withTransaction()` keeps multi-order updates atomic.
 - **Typed error responses** through a shared `HttpError` + `asyncHandler` pattern.
 - **25/25 integration tests** running against an in-memory Postgres (`pg-mem`), so no database is required to verify the API.
- Known interface difference to reconcile**

The frontend's stage list contains eight stages including `atCustomerCity`, while the server's `DELIVERY_STAGES` models a seven-stage band. Any wiring work must map the client's extra stage onto the server model — the server README already records this discrepancy.

10 · Current Limitations

An honest list of what is *not* yet in place, so the demo is not over-sold.

Area	Status
Frontend ↔ backend wiring	The SPA has no API client. All reads/writes are <code>localStorage</code> ; the Express service is not called by the UI.
Authentication strength	Demo-grade: passwords are stored in plaintext in <code>localStorage</code> , and OTPs are displayed on screen rather than sent by SMS.
Real payments	Simulated. The payment modal verifies a generated demo OTP; no gateway is integrated.
User-generated content safety	Several views interpolate user input straight into <code>innerHTML</code> (search results in particular), which is a DOM-XSS exposure.
Robustness edges	<code>renderDemand()</code> assumes a non-empty demand object; <code>getUser()</code> re-parses <code>localStorage</code> on every call; the auction <code>tickTimers()</code> interval is never cleared.
Money reporting	"Total Spent" currently includes refunded orders.
Accessibility	Modals lack Escape-to-close and a focus trap; most overlays do not restore focus on close.
Third-party weight	Leaflet, Google Fonts and map tile/geocoding services are loaded from the network, so the app needs connectivity for the map view.
Build tooling	No package manifest, bundler, linter or frontend test suite at the repository root.

11 · Feature Checklist

Module	Capabilities present
Auth & roles	3 roles · role switch · username/phone login · signup with OTP · role-aware profile fields · demo shortcut · logout.
Farmer	Dashboard stats · listing CRUD with photo compression · auctions & live bidding · market demand chart · AI advisor · pending/completed sales · money ledger · profile & settings.
Consumer	Dashboard · nearby grid · search (produce + farmers) · quantity picker · cart drawer · checkout with demo payment OTP · purchased items · history · tracking modal · follow · rate · donate · notifications · money ledger · profile & settings.
FPO	Logistics dashboard · Tier-1 weightment + AGMARK grading · Tier-2 LTL corridor batching with capacity meter · Tier-3 hub receive, slot view & EV dispatch · profile & settings.
Tracking	8-stage shared pipeline · vertical timeline · Leaflet map · route HUD · delivery-slot scheduling · live cross-role updates via events.
Platform	3 languages · 3 palettes · light/dark · responsive layout · toasts · animations & audio · empty states · reduced-motion support.
Backend	JWT auth · bcrypt · OTP codes · produce/auctions/orders/fpo routes · transactional FPO batch attachment · 25 passing integration tests.